Open in app ↗ Sign up Sign in

Medium Search

Scaling DuckDB: A Modern Architecture for Analytical Data Applications

4 min read · Mar 31, 2025

Gagan Taneja Follow

Listen Share

While attending a large Data conference I was hearing a consistent theme: data cluster headaches. Whether it's poor, inconsistent performance, outrageous costs, noisy neighbors or the unfortunate combination of all, it's clear that the current paradigm isn't working optimally for everyone.

A few months ago, I discovered DuckDB, a remarkably lightweight solution. Its blazing-fast vectorized query engine offers a compelling alternative to traditional, heavyweight data clusters centric approach. You can learn more about DuckDB here:

While DuckDB is well-known for its outstanding single-node performance, its capabilities extend far beyond. Let's move past the misconception that it's limited to single-node operations and explore both its single-node advantages and the ways in which horizontal scaling can unlock even greater potential.

Single-Node Performance

DuckDB consistently dominates single-node performance benchmarks, and this isn't by chance. Its design meticulously leverages SIMD instructions and CPU L1/L2 caches. Furthermore, by avoiding code generation and JIT compilation during cold starts, DuckDB achieves significant speed advantages. A quick search online will reveal benchmarks performed by various organizations that support this claim.

While I understand the skepticism surrounding benchmarks — and ultimately, your own testing is crucial — my personal experience with reasonably sized datasets

shows DuckDB consistently outperforming Apache Spark by a factor of 3–8x.

time	tag	engine
int128	varchar	varchar
6653	basic aggregation	duckdb
11641	basic aggregation	spark
896	filter_with_aggregation	duckdb
4348	filter_with_aggregation	spark
6033	grouping_set	duckdb
14211	grouping_set	spark
4168	join_and_aggregation	duckdb
28947	join_and_aggregation	spark
25222	very_large_distinct	duckdb
275090	very_large_distinct	spark
10 rows		3 columns

Spark Vs DuckDB benchmark

You can get more information about my testing here. You will be able to update it based on your dataset and queries which you would like to benchmark.

Here is another link providing information about how DuckDBs own benchmarks have improved over a period of time

Modern Hardware: A Game Changer for Traditional Clusters

The sheer power of modern hardware is transforming the data landscape. CPUs with 96–128 cores (192–256 threads) are becoming commonplace. A dual-socket servers can boast an impressive 384–512 threads. This single-server horsepower can now handle workloads that were once exclusively reserved for distributed clusters.

Moreover, network bandwidth has exploded, surging from 1 Gbps to 100 Gbps. This enables a single node to theoretically ingest 70–80 Gbps of data per second.

Get Gagan Taneja's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

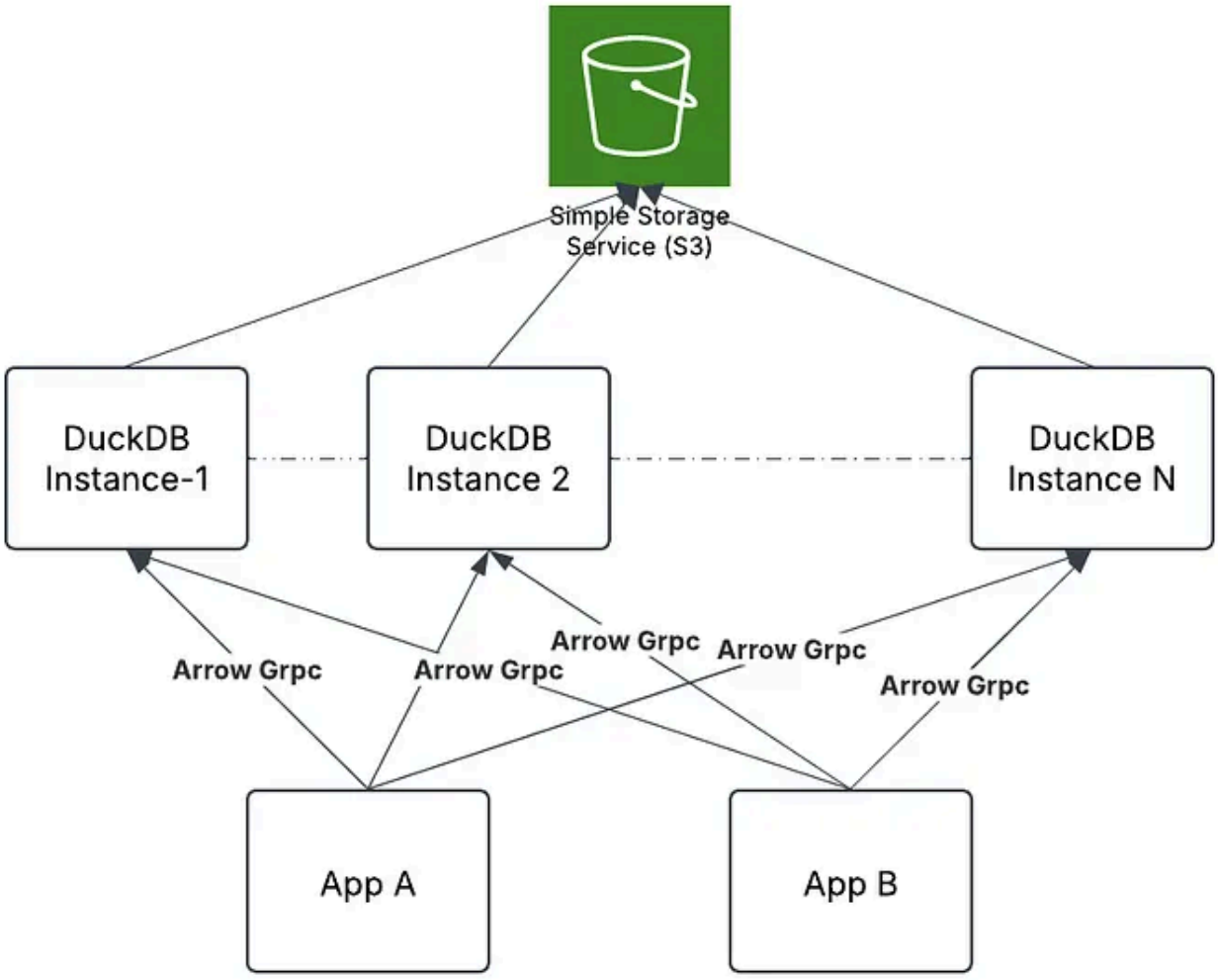
Subscribe

Remember me for faster sign in

Historically, limitations in both compute and network bandwidth necessitated cluster-based computing. However, these limitations are largely obsolete today. In fact, leading providers of cluster-based query engines often recommend core counts per cluster that can now be readily satisfied by a single, powerful node.

Scaling DuckDB with a Distributed Architecture

To overcome DuckDB's single-node limitations, I present a distributed architecture that enables scaling to handle larger workloads.



Reference Architecture

Cluster Architecture:

- The “DuckDB cluster” consists of a collection of independent DuckDB instances.
- These instances operate autonomously, allowing for simple and rapid scaling based on query load demands.

Application Integration:

- Existing and new applications continue using familiar SQL/JDBC interfaces and connection pools (typically 10–20 connections).
- These connections are distributed across multiple DuckDB instances.
- With data formats like Delta Lake and Iceberg providing consistency, each DuckDB instance sees a consistent view of the data.
- Query load is distributed across these instances, which can be dynamically scaled up or down based on application demand.

Protocol:

- The network protocol between the JDBC driver and the DuckDB instances utilizes Arrow Flight over gRPC for efficient data transfe.

Benefits:

- **Minimal Application Changes:** Requires only updating the JDBC driver JAR.
- **Enhanced Performance:** Leverages DuckDB’s speed and avoids the overhead of data movement within a traditional cluster.
- **Simplified Architecture:** Facilitates modern deployment practices and faster scaling.

Limitations:

- **Very Large Queries:** DuckDB doesn’t natively distribute a single query across multiple nodes.
- **Mitigation:** This limitation is often mitigated by pre-aggregating large datasets, a common practice regardless of the underlying system. For scenarios requiring analysis of massive data volumes, a large query can be decomposed into smaller,

independent queries executed on different nodes, with the results then unioned together.

Open source Implementation:

You can get the implementation over here with JDBC implementation and load balancing

Final Thoughts: A Call to Innovation

Given the significant advancements in server core density and network bandwidth, replacing a traditional cluster with a single DuckDB instance is increasingly becoming a viable and compelling alternative for a growing number of deployments. The ultimate success of your data initiatives depends on embracing architectural innovation, rather than blindly adhering to outdated or standardized approaches.

I am actively seeking contributions to further develop and solidify the code and architectures outlined here. If you're interested in contributing, please reach out, or submit a pull request to the code repository linked within this article.



Follow

Written by Gagan Taneja

23 followers · 9 following